

Lesson 17 Structured Note: Coding Infix-to-Postfix Conversion Using a Linked-List Stack

0. Current Position in the Course

You are now in **Lesson 17** of the stack curriculum.

Before Lesson 17, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Bracket matching was extended to <code>()</code> , <code>[]</code> , and <code>{}</code> .
Lesson 13	You learned infix, prefix, postfix, precedence, and simple manual expression conversion.
Lesson 14	You learned associativity and unary operators.
Lesson 15	You learned the logic for converting infix to postfix using a stack.

Lesson	Main idea
Lesson 16	You wrote an infix-to-postfix program using an array character stack.

Lesson 17 continues:

Phase 4: Expression Notation and Conversion

Lesson 16 taught the program version using an **array stack**.

Lesson 17 teaches the same conversion idea, but now the operator stack is implemented using a **linked list**.

In simple words:

- Lesson 15 taught the rule.
- Lesson 16 coded the rule using an array stack.
- Lesson 17 codes the rule using a linked-list stack.

1. Lesson 17 Goal

By the end of this lesson, you should understand:

- What infix-to-postfix conversion means.
- Why postfix notation is useful.
- Why a stack is used during conversion.
- How the Lesson 17 version differs from Lesson 16.
- How to use a linked-list stack for operators.
- Why the linked-list stack stores `char` values.
- How `struct Node` is used for the stack.
- What the global `top` pointer means.
- How `push(char x)` works.
- How `pop()` works.
- How `stackTop()` works.
- How `pre(char x)` gives operator precedence.
- How `isOperand(char x)` separates operands from operators.
- Why the dummy symbol `#` is pushed first.
- Why `#` should not appear in the final postfix answer.
- How the `InToPost()` function scans the infix expression.
- Why the variables `i` and `j` are separate.
- Why `i` does not always move forward immediately.
- How to dry-run `A+B*C`.
- How to dry-run a case where popping happens before pushing.
- The time complexity of the algorithm.

- The space complexity of the algorithm.
- The beginner limitations of this version.
- Common mistakes to avoid.

2. Quick Review: What Is an Expression?

An **expression** is a combination of operands and operators.

Example:

A + B

This expression has:

Symbol	Role
A	Operand
+	Operator
B	Operand

3. Operand

An **operand** is a value or variable used in an expression.

Examples:

A, B, C, x, y, 5, 9

In this expression:

A + B

The operands are:

A and B

They are the values being used by the operator.

4. Operator

An **operator** is a symbol that performs an operation.

Common operators are:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

In this expression:

A + B

The operator is:

+

Because

+

 tells us to add

A

 and

B

.

5. Quick Review: Infix and Postfix

Lesson 17 focuses on converting from **infix** to **postfix**.

5.1 Infix Notation

In **infix notation**, the operator is written between the operands.

Structure:

Operand Operator Operand

Example:

A + B

The operator **+** is between **A** and **B**, so this is infix.

Humans usually write expressions in infix form.

Examples:

A + B
A - B
A * B
A / B
A + B * C

5.2 Postfix Notation

In **postfix notation**, the operator is written after the operands.

Structure:

Operand Operand Operator

Example:

A B +

This means the same thing as:

A + B

The operator comes after the operands, so this is postfix.

Examples:

Infix	Postfix
A + B	AB+
A - B	AB-

Infix	Postfix
A * B	AB*
A / B	AB/

Important idea:

Infix and postfix can represent the same expression. The meaning does not change. Only the position of the operator changes.

6. What Problem Are We Solving?

We want to convert an expression from infix form to postfix form.

Example:

A+B*C

The postfix form is:

ABC*+

Why?

Because multiplication has higher precedence than addition.

So:

A+B*C

really means:

A+(B*C)

In postfix:

B*C

becomes:

BC*

Then we add A :

ABC*+

7. Why Do We Need a Stack?

In infix expressions, operators do not always execute in the order we see them.

Example:

A+B*C

When scanning from left to right, we see + before *.

But * must happen first because multiplication has higher precedence.

So the + operator must wait.

The stack is used as a waiting area for operators.

Simple idea:

Operands go directly to postfix.
Operators wait in the stack.

The stack is not the final answer.

The postfix array is the final answer.

The stack only helps us decide when operators should be moved to the answer.

8. Review of the Main Infix-to-Postfix Rule

When scanning the infix expression from left to right:

If the current symbol is an operand:
copy it directly to postfix

If the current symbol is an operator:
compare it with the operator on top of the stack

If the current operator has higher precedence:
push it onto the stack

If the current operator has lower or equal precedence:
pop from the stack into postfix

At the end:

Pop all remaining operators from the stack into postfix.

9. What Changes in Lesson 17?

Lesson 16 used an array stack.

Lesson 17 uses a linked-list stack.

Lesson 16 Array Stack

```
struct Stack {  
    int size;  
    int top;  
    char *S;  
};
```

In the array version:

top is an index.

Example:

top = 2

This means the top character is stored at array index **2**.

Lesson 17 Linked-List Stack

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

In the linked-list version:

top is a pointer.

Example:

top -> '*' -> '+' -> '#' -> NULL

Here, `top` points to the first node.

The first node contains `*`.

So `*` is the top of the stack.

10. Array Stack vs Linked-List Stack in This Lesson

Concept	Array stack version	Linked-list stack version
Storage	Character array	Linked nodes
Top meaning	Index	Pointer
Empty condition	<code>top == -1</code>	<code>top == NULL</code>
Push action	Store in next array slot	Create a new node
Pop action	Decrease top index	Remove and free top node
Memory	Fixed size	Grows using heap memory
Used in	Lesson 16	Lesson 17

The stack rule does not change.

Both versions still follow:

LIFO: Last In, First Out

11. Why Use a Linked-List Stack?

A linked-list stack grows by creating nodes dynamically.

That means every time we push an operator, we create a new node using:

```
malloc()
```

Every time we pop an operator, we remove a node using:

```
free()
```

So the stack can grow as needed, as long as heap memory is available.

Important idea:

```
Array stack uses fixed storage.  
Linked-list stack creates nodes when needed.
```

12. Why the Stack Stores Characters

In previous stack lessons, the stack often stored integers such as:

```
10, 20, 30
```

But in infix-to-postfix conversion, the stack stores expression symbols such as:

```
+, -, *, /, #
```

These are characters.

So the node stores:

```
char data;
```

not:

```
int data;
```

13. Node Structure

The linked-list stack is made of nodes.

Each node stores two things:

Field	Meaning
data	Stores the character, such as <code>+</code> or <code>*</code> .
next	Points to the next node below it.

Code:

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

Example node:

```
[ data = '+' | next ]
```

If the stack has several operators, it may look like this:

```
top -> '*' -> '+' -> '#' -> NULL
```

14. The Global `top` Pointer

Lesson 17 uses a global pointer:

```
struct Node *top = NULL;
```

This means the stack starts empty.

Empty stack:

```
top -> NULL
```

After pushing some characters, the stack may look like this:

```
top -> '*' -> '+' -> '#' -> NULL
```

Here:

- `top` points to `*`.
- `*` is the current top operator.
- `+` is below `*`.
- `#` is at the bottom.

15. Empty Condition in the Linked-List Stack

A linked-list stack is empty when:

```
top == NULL
```

Why?

Because `top` is a pointer.

If `top` points to nothing, there is no node in the stack.

Example:

```
top -> NULL
```

This means the stack has no elements.

16. The `push(char x)` Function

16.1 Meaning

`push(char x)` inserts a character at the top of the linked-list stack.

Example before pushing `*`:

```
top -> '+' -> '#' -> NULL
```

Call:

```
push('*');
```

After pushing:

```
top -> '*' -> '+' -> '#' -> NULL
```

The new character becomes the top.

16.2 Code

```
void push(char x) {
    struct Node *t;
    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
        exit(1);
    }

    t->data = x;
    t->next = top;
    top = t;
}
```

16.3 Explanation of `push(char x)`

Step 1: Create a temporary pointer.

```
struct Node *t;
```

t will point to the new node.

Step 2: Allocate memory for one node.

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This creates one new node in heap memory.

Step 3: Check if memory allocation failed.

```
if (t == NULL)
```

If malloc() returns NULL, no memory was available.

That means stack overflow.

Step 4: Store the character.

```
t->data = x;
```

If we call:

```
push('+');
```

then:

```
t->data = '+';
```

Step 5: Link the new node to the old top.

```
t->next = top;
```

This makes the new node point to the previous top node.

Step 6: Move top to the new node.

```
top = t;
```

Now the new node is the top of the stack.

17. The Most Important Push Pattern

The key linked-list push pattern is:

```
t->next = top;  
top = t;
```

Simple meaning:

1. The new node points to the old top.
2. `top` moves to the new node.

The order matters.

Correct order:

```
t->next = top;  
top = t;
```

Wrong order:

```
top = t;  
t->next = top;
```

The wrong order can damage the stack.

It may lose the old stack or make the new node point to itself.

18. The `pop()` Function

18.1 Meaning

`pop()` removes and returns the top character from the stack.

Example before pop:

```
top -> '*' -> '+' -> '#' -> NULL
```

Call:

```
pop();
```

Returned value:

```
*
```

After pop:

```
top -> '+' -> '#' -> NULL
```

18.2 Code

```
char pop() {
    struct Node *p;
    char x = '\0';

    if (top == NULL) {
        printf("Stack Underflow\n");
        return x;
    }

    p = top;
    x = p->data;
    top = top->next;
    free(p);

    return x;
}
```

18.3 Explanation of pop()

Step 1: Create a temporary pointer.


```
struct Node *p;
```

`p` will point to the node that will be removed.

Step 2: Create a return variable.

```
char x = '\0';
```

`x` stores the popped character.

If the stack is empty, `x` stays as `\0`.

Step 3: Check underflow.

```
if (top == NULL)
```

If `top == NULL`, the stack is empty.

There is no node to remove.

This is called stack underflow.

Step 4: Save the old top node.

```
p = top;
```

Now `p` points to the node that will be deleted.

Step 5: Save the character.

```
x = p->data;
```

This saves the top character so we can return it after deleting the node.

Step 6: Move `top` to the next node.

```
top = top->next;
```

Now the second node becomes the new top.

Step 7: Free the old top node.

```
free(p);
```

This releases the memory used by the removed node.

Step 8: Return the saved character.

```
return x;
```

19. The Most Important Pop Pattern

The key linked-list pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Simple meaning:

1. Save the old top node.
2. Save its character.
3. Move `top` to the next node.
4. Free the old node.
5. Return the saved character.

Do not free the node before moving `top` safely.

Wrong idea:

```
free(top);  
top = top->next;
```

This is unsafe because after freeing `top`, using `top->next` is invalid.

20. The `stackTop()` Function

20.1 Meaning

`stackTop()` reads the top character without removing it.

It is like looking at the top of the stack.

Example:

```
top -> '*' -> '+' -> '#' -> NULL
```

Call:

```
stackTop();
```

It returns:

```
*
```

But the stack does not change.

20.2 Code

```
char stackTop() {  
    if (top == NULL)  
        return '\0';  
  
    return top->data;  
}
```

20.3 Why `stackTop()` Is Needed

During infix-to-postfix conversion, we often need to compare:

```
current operator
```

with:

operator on top of the stack

So we need `stackTop()` to check the top operator without removing it.

Example:

```
if (pre(infix[i]) > pre(stackTop()))
```

This compares the precedence of the current operator with the precedence of the stack-top operator.

21. Operator Precedence

Precedence means priority.

It answers this question:

Which operator should be handled first?

For this beginner version:

Operator	Precedence
<code>*</code> , <code>/</code>	2
<code>+</code> , <code>-</code>	1
<code>#</code>	0

Multiplication and division have higher precedence than addition and subtraction.

So:

$A+B*C$

means:

$A+(B*C)$

not:

(A+B)*C

22. The `pre(char x)` Function

22.1 Meaning

`pre(char x)` returns the precedence number of an operator.

Code:

```
int pre(char x) {  
    if (x == '+' || x == '-')  
        return 1;  
    else if (x == '*' || x == '/')  
        return 2;  
  
    return 0;  
}
```

22.2 Explanation

If `x` is `+` or `-`, return `1`:

```
if (x == '+' || x == '-')  
    return 1;
```

If `x` is `*` or `/`, return `2`:

```
else if (x == '*' || x == '/')  
    return 2;
```

For anything else, return `0`:

```
return 0;
```

This is useful for the dummy `#`, because:

```
pre('#') = 0
```

23. The `isOperand(char x)` Function

23.1 Meaning

`isOperand(char x)` checks whether a character is an operand.

For this beginner version:

+ - * / are operators.
Everything else is treated as an operand.

Code:

```
int isOperand(char x) {  
    if (x == '+' || x == '-' || x == '*' || x == '/')  
        return 0;  
  
    return 1;  
}
```

23.2 Return Values

Return value	Meaning
1	It is an operand.
0	It is an operator.

Examples:

```
isOperand('A') returns 1  
isOperand('B') returns 1  
isOperand('5') returns 1  
isOperand('+') returns 0  
isOperand('*') returns 0
```

23.3 Important Limitation

This beginner version is simple.

It treats anything that is not `+`, `-`, `*`, or `/` as an operand.

So it works well for simple expressions like:

```
A+B*C
```

But it does not properly handle:

- spaces
- parentheses
- multi-digit numbers
- unary minus
- power operator `^`
- functions like `log()`

Those are advanced improvements.

24. The Dummy `#` Symbol

Lesson 17 introduces an important trick:

```
push( '#' );
```

This pushes `#` onto the stack before scanning the infix expression.

So the stack starts like this:

```
top -> '#' -> NULL
```

The `#` symbol is called a dummy symbol or sentinel.

It is not a real operator from the expression.

It is only used to mark the bottom of the stack.

25. Why Do We Use #?

The dummy # helps prevent empty-stack comparison problems.

Without #, before comparing precedence, we might need to ask:

Is the stack empty?

But with #, the stack is not empty during the main comparison.

Since:

$\text{pre}(\text{'\#'}) = 0$

any real operator has greater precedence than #.

Example:

$\text{pre}(\text{'+'}) > \text{pre}(\text{'\#'})$

This means:

$1 > 0$

So + can be pushed safely.

Another example:

$\text{pre}(\text{'*'}) > \text{pre}(\text{'\#'})$

This means:

$2 > 0$

So * can also be pushed safely.

26. Important Rule About

The dummy # should not be part of the final postfix expression.

Correct postfix for:

A+B*C

is:

ABC*+

Incorrect raw output if # is copied:

ABC*+**#**

or:

ABC*+**#**

The correct program should stop popping when the stack top is #, then remove # separately.

Correct idea:

```
while (stackTop() != '#') {  
    postfix[j++] = pop();  
}  
  
pop(); // remove '#'
```

This removes # but does not add it to postfix.

27. The InToPost() Function

27.1 Meaning

InToPost() converts an infix expression into a postfix expression.

Example input:

```
A+B*C
```

Output:

```
ABC*+
```

The function receives an infix string and returns a new postfix string.

27.2 Main Variables

Inside `IntoPost()`, we use:

```
int i = 0;  
int j = 0;
```

Meaning:

Variable	Purpose
<code>i</code>	Reads characters from the infix expression.
<code>j</code>	Writes characters into the postfix expression.

Example:

```
infix[i]
```

means:

```
current character from infix
```

And:

```
postfix[j]
```

means:

current writing position in postfix

28. Why `i` and `j` Are Separate

`i` and `j` are separate because reading and writing do not always happen at the same speed.

Sometimes the program reads one input character and writes one output character.

Example:

A

A is an operand, so it goes directly to postfix.

In this case:

```
i moves forward  
j moves forward
```

But sometimes the program pops an operator from the stack without moving forward in the infix expression.

Example:

A*B+C

When the program reaches `+`, the stack top is `*`.

Since `*` has higher precedence than `+`, we must pop `*` into postfix first.

But we have not finished handling `+` yet.

So:

```
j moves forward because we wrote '*'  
i does not move because '+' still needs to be handled
```

This is one of the most important ideas in Lesson 17.

29. Why We Use a `while` Loop Style

In infix-to-postfix conversion, the current input character is not always consumed immediately.

When the current operator has lower or equal precedence than the stack top, we pop from the stack.

But we do not move `i` forward yet.

That means the same input operator must be checked again.

This is why a `while` loop is useful.

It gives us control over when `i` increases.

Example:

```
if (isOperand(infix[i])) {  
    postfix[j++] = infix[i++];  
}
```

Here, `i` moves because the operand has been fully handled.

But here:

```
postfix[j++] = pop();
```

`i` does not move because the current operator still has not been pushed or finished.

30. Main Conversion Algorithm in Simple Words

To convert infix to postfix using a linked-list stack:

1. Create an empty linked-list stack.
2. Push the dummy symbol `#` onto the stack.
3. Create a postfix array.
4. Start scanning the infix expression from left to right.
5. If the current symbol is an operand, copy it to postfix.
6. If the current symbol is an operator, compare its precedence with `stackTop()`.
7. If the current operator has higher precedence, push it.
8. If the current operator has lower or equal precedence, pop from the stack into postfix.
9. After the infix expression ends, pop all operators until `#` is reached.

10. Remove '#' from the stack.
 11. Add '\0' at the end of the postfix string.
 12. Return the postfix expression.
-

31. Main Conversion Algorithm as Pseudocode

```
InToPost(infix):
    i = 0
    j = 0
    create postfix array
    push '#'

    while infix[i] is not end of string:
        if infix[i] is operand:
            postfix[j] = infix[i]
            i++
            j++
        else:
            if precedence(infix[i]) > precedence(stackTop()):
                push(infix[i])
                i++
            else:
                postfix[j] = pop()
                j++

    while stackTop() is not '#':
        postfix[j] = pop()
        j++

    pop '#' from stack
    postfix[j] = '\0'
    return postfix
```

32. Complete Beginner C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char data;
    struct Node *next;
```

```

};

struct Node *top = NULL;

void push(char x) {
    struct Node *t;
    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
        exit(1);
    }

    t->data = x;
    t->next = top;
    top = t;
}

char pop() {
    struct Node *p;
    char x = '\0';

    if (top == NULL) {
        printf("Stack Underflow\n");
        return x;
    }

    p = top;
    x = p->data;
    top = top->next;
    free(p);

    return x;
}

char stackTop() {
    if (top == NULL)
        return '\0';

    return top->data;
}

int pre(char x) {
    if (x == '+' || x == '-')
        return 1;
    else if (x == '*' || x == '/')
        return 2;
}

```

```

        return 0;
    }

    int isOperand(char x) {
        if (x == '+' || x == '-' || x == '*' || x == '/')
            return 0;

        return 1;
    }

    char *InToPost(char *infix) {
        int i = 0;
        int j = 0;
        int len = strlen(infix);

        char *postfix = (char *)malloc((len + 2) * sizeof(char));

        if (postfix == NULL) {
            printf("Memory allocation failed\n");
            return NULL;
        }

        push('#');

        while (infix[i] != '\0') {
            if (isOperand(infix[i])) {
                postfix[j++] = infix[i++];
            } else {
                if (pre(infix[i]) > pre(stackTop())) {
                    push(infix[i++]);
                } else {
                    postfix[j++] = pop();
                }
            }
        }

        while (stackTop() != '#') {
            postfix[j++] = pop();
        }

        pop(); // remove dummy '#'

        postfix[j] = '\0';

        return postfix;
    }

    int main() {

```

```
char infix[] = "A+B*C";

char *postfix = InToPost(infix);

if (postfix != NULL) {
    printf("Infix: %s\n", infix);
    printf("Postfix: %s\n", postfix);
    free(postfix);
}

return 0;
}
```

Expected output:

```
Infix: A+B*C
Postfix: ABC*+
```

33. Why `malloc((len + 2) * sizeof(char))` Is Used

The postfix result needs memory.

If the infix expression has length `len`, the postfix expression will usually need about the same number of visible characters.

But we also need space for:

1. The null terminator `\0`.
2. Extra safety space because the lesson uses a dummy `#` internally.

So this line is used:

```
char *postfix = (char *)malloc((len + 2) * sizeof(char));
```

Simple meaning:

```
Create enough space for the postfix answer.
```


34. Why We Add `\0` at the End

In C, strings must end with the null terminator:

```
'\0'
```

So after building the postfix expression, we write:

```
postfix[j] = '\0';
```

This tells C:

```
The postfix string ends here.
```

Without `\0`, printing the string with `printf("%s", postfix)` may produce garbage output.

35. Dry Run 1: `A+B*C`

Expression:

```
A+B*C
```

Expected postfix:

```
ABC*+
```

Initial state:

```
Stack: #  
Postfix: empty  
i = 0  
j = 0
```

For this table, the stack is shown from bottom to top.

Step	Current symbol	Action	Stack	Postfix
Start	—	Push dummy <code>#</code>	<code>#</code>	empty

Step	Current symbol	Action	Stack	Postfix
1	A	Operand, copy to postfix	#	A
2	+	pre(+) > pre(#), push +	# +	A
3	B	Operand, copy to postfix	# +	AB
4	*	pre(*) > pre(+), push *	# + *	AB
5	C	Operand, copy to postfix	# + *	ABC
End	—	Pop *	# +	ABC*
End	—	Pop +	#	ABC*+
End	—	Remove #	empty	ABC*+

Final answer:

A+B*C -> ABC*+

36. Dry Run 1 Explanation in Simple Words

The expression is:

A+B*C

A is an operand, so it goes directly to postfix.

+ is an operator, and the stack only has #, so + is pushed.

B is an operand, so it goes directly to postfix.

* is an operator. It has higher precedence than +, so it is pushed above +.

C is an operand, so it goes directly to postfix.

Now the input is finished.

So we pop the remaining real operators:

First *, then +.

The final postfix expression is:

ABC*+

37. Dry Run 2: A*B+C

This example shows why **i** does not always move forward.

Expression:

A*B+C

Expected postfix:

AB*C+

Initial state:

Stack: #
Postfix: empty

Step	Current symbol	Action	Stack	Postfix	Does i move?
Start	—	Push dummy #	#	empty	—
1	A	Operand, copy to postfix	#	A	Yes
2	*	<code>pre(*) > pre(#)</code> , push *	# *	A	Yes
3	B	Operand, copy to postfix	# *	AB	Yes
4	+	<code>pre(+) <= pre(*)</code> , pop *	#	AB*	No
5	+	Now <code>pre(+) > pre(#)</code> , push +	# +	AB*	Yes
6	C	Operand, copy to postfix	# +	AB*C	Yes
End	—	Pop +	#	AB*C+	—
End	—	Remove #	empty	AB*C+	—

Final answer:

$A*B+C \rightarrow AB*C+$

Important point:

When $+$ first appears, $*$ is on top of the stack.

Since $*$ has higher precedence, $*$ must be popped first.

But $+$ has not been handled yet.

So i does not move during that pop.

38. Dry Run 3: $A+B-C$

This example shows equal precedence.

Expression:

$A+B-C$

Expected postfix:

$AB+C-$

Why?

$+$ and $-$ have equal precedence.

For these operators, we handle left to right.

So:

$A+B-C$

means:

$(A+B)-C$

Dry run:

Step	Current symbol	Action	Stack	Postfix
Start	—	Push #	#	empty
1	A	Operand	#	A
2	+	Push +	# +	A
3	B	Operand	# +	AB
4	-	<code>pre(-) <= pre(+)</code> , pop +	#	AB+
5	-	Now push -	# -	AB+
6	C	Operand	# -	AB+C
End	—	Pop -	#	AB+C-
End	—	Remove #	empty	AB+C-

Final answer:

A+B-C -> AB+C-

39. Why Lower or Equal Precedence Causes Pop

The program uses this idea:

```
If current operator has higher precedence than stack top:
    push current operator
else:
    pop stack top into postfix
```

Why?

Because if the stack-top operator has higher or equal priority, it should appear in postfix before the current operator.

Example:

A*B+C

When we reach +, the stack has *.

`*` has higher precedence than `+`.

So `*` must be moved to postfix first.

Another example:

```
A+B-C
```

When we reach `-`, the stack has `+`.

`+` and `-` have equal precedence.

Because they are handled left to right, `+` must be moved to postfix before `-`.

40. What Happens After the Infix String Ends?

After the loop finishes scanning all input characters, some operators may still be waiting in the stack.

Example:

```
Stack: # + *  
Postfix: ABC
```

The expression has ended, so now it is safe to move all remaining real operators into postfix.

We do:

```
while (stackTop() != '#') {  
    postfix[j++] = pop();  
}
```

This pops operators until `#` is reached.

Then we remove `#`:

```
pop();
```

But we do not copy `#` into postfix.

41. Why # Should Be Removed Separately

If we write:

```
while (top != NULL) {  
    postfix[j++] = pop();  
}
```

then # may be added to postfix.

That is not correct.

The dummy # was only used to mark the bottom of the stack.

It is not part of the expression.

Correct logic:

```
while (stackTop() != '#') {  
    postfix[j++] = pop();  
}  
  
pop(); // remove # only
```

This keeps the final postfix answer clean.

42. Memory Cleanup

This lesson uses dynamic memory in two places:

1. Nodes created by `push()` using `malloc()`.
2. The postfix array created using `malloc()`.

Nodes are cleaned up when they are popped:

```
free(p);
```

The postfix array should be cleaned up in `main()` after it is printed:

```
free(postfix);
```

This prevents memory leaks.

A memory leak happens when memory is allocated but never released.

43. Time Complexity

The time complexity is:

```
O(n)
```

where **n** is the number of characters in the infix expression.

Why?

Because each character is scanned from left to right.

Each operator is pushed at most once.

Each operator is popped at most once.

So the total work grows linearly with the input size.

Example:

If the expression has 5 characters, we do a small amount of work.

If the expression has 50 characters, we do about 10 times more work.

That is linear time.

44. Space Complexity

The space complexity is:

```
O(n)
```

Why?

Because we need extra memory for:

1. The linked-list operator stack.
2. The postfix output array.

In the worst case, the stack and output can grow with the input length.

So the space used depends on `n`.

45. What This Beginner Version Can Handle

This Lesson 17 version can handle simple expressions with:

- single-character operands
- operators `+`, `-`, `*`, `/`
- no spaces
- no parentheses

Examples it can handle:

```
A+B*C
A*B+C
A+B-C
A*B/C
A+B*C-D/E
```

46. What This Beginner Version Cannot Handle Yet

This version does not properly handle:

- parentheses, such as `(A+B)*C`
- spaces, such as `A + B`
- multi-digit numbers, such as `12+34`
- unary minus, such as `-A+B`
- power operator, such as `A^B`
- functions, such as `log(A)`
- variables with more than one character, such as `total+cost`

For now, use simple expressions like:

```
A+B*C
```

not advanced expressions like:

```
(A+12)*log(B)
```

47. Common Beginner Mistakes

Mistake 1: Thinking `top` is an index

In Lesson 17, `top` is not an index.

It is a pointer.

Correct idea:

```
top points to the first node.
```

Mistake 2: Forgetting to push `#`

If you forget:

```
push( '#' );
```

then `stackTop()` may be called on an empty stack.

The dummy `#` helps avoid that problem.

Mistake 3: Adding `#` to the postfix answer

Wrong output:

```
ABC*+#
```

Correct output:

```
ABC*+
```

The dummy `#` must be removed, not copied to postfix.

Mistake 4: Moving `i` after every pop

When the program pops an operator because of precedence, the current input operator has not been handled yet.

So `i` should not move.

Wrong idea:

```
pop and move i
```

Correct idea:

```
pop, write to postfix, keep i where it is
```

Mistake 5: Forgetting the null terminator

The postfix string must end with:

```
'\0'
```

Correct:

```
postfix[j] = '\0';
```

Without this, printing the string may produce garbage.

Mistake 6: Forgetting to free memory

When a node is popped, it should be freed:

```
free(p);
```

When the postfix result is no longer needed, it should be freed:

```
free(postfix);
```

Mistake 7: Expecting this version to handle parentheses

This beginner version does not handle parentheses.

Expression:

```
(A+B)*C
```

needs extra rules for `(` and `)`.

That is not included in this Lesson 17 beginner code.

48. Full Code Flow Summary

The program has these main parts:

1. Include header files.
2. Define `struct Node`.
3. Declare global `top` pointer.
4. Write `push()`.
5. Write `pop()`.
6. Write `stackTop()`.
7. Write `pre()`.
8. Write `isOperand()`.
9. Write `InToPost()`.
10. Test the function in `main()`.
11. Free the postfix memory.

49. Header Files Used

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Meaning:

Header	Used for
<code>stdio.h</code>	<code>printf()</code>
<code>stdlib.h</code>	<code>malloc()</code> , <code>free()</code> , <code>exit()</code>
<code>string.h</code>	<code>strlen()</code>

50. Function Summary Table

Function	Purpose	Changes stack?
<code>push(char x)</code>	Adds an operator to the stack.	Yes
<code>pop()</code>	Removes and returns the top operator.	Yes
<code>stackTop()</code>	Reads the top operator without removing it.	No
<code>pre(char x)</code>	Returns precedence of an operator.	No
<code>isOperand(char x)</code>	Checks if a character is an operand.	No
<code>InToPost(char *infix)</code>	Converts infix to postfix.	Yes, through push/pop

51. Mental Model

Think of the conversion like this:

The infix expression is the input.
The postfix array is the answer sheet.
The linked-list stack is the waiting room for operators.
The dummy # is the floor of the waiting room.

Operands do not need to wait.

They go directly to the answer.

Operators may need to wait because precedence decides their correct order.

52. Lesson 17 Final Summary

Lesson 17 teaches how to convert infix expressions to postfix expressions using a linked-list stack.

The main expression example is:

A+B*C

The correct postfix result is:

ABC*+

The main rules are:

Operand -> copy directly to postfix
Operator -> compare with stack top
Higher precedence -> push
Lower or equal precedence -> pop
End of input -> pop remaining real operators
Dummy # -> marks bottom of stack
Final # -> remove, do not print

The main linked-list stack idea is:

Each pushed operator becomes a new node.
Each popped operator removes the top node.
top points to the first node.

The time complexity is:

$O(n)$

The space complexity is:

$O(n)$

53. Practice Tasks

Task 1

Convert this infix expression to postfix manually:

A+B*C

Expected answer:

ABC*+

Task 2

Convert this expression:

A*B+C

Expected answer:

AB*C+

Task 3

Convert this expression:

A+B-C

Expected answer:

AB+C-

Task 4

Explain why the dummy # is used.

Answer idea:

marks the bottom of the stack and prevents empty-stack comparison problems.

Task 5

Explain why `#` should not appear in the final postfix expression.

Answer idea:

`#` is not a real operator from the expression. It is only a helper symbol.

Task 6

In this expression:

`A*B+C`

When the program reaches `+`, why does it pop `*` first?

Answer idea:

Because `*` has higher precedence than `+`, so `*` must appear in postfix before `+`.

Task 7

What is the difference between `i` and `j`?

Answer idea:

`i` scans the infix expression.
`j` writes into the postfix expression.

54. Quick Revision Checklist

Before moving to the next lesson, make sure you can answer these questions:

- What is infix notation?
- What is postfix notation?
- Why do we use a stack for conversion?
- What does the linked-list stack store?

- What does `top` mean in this lesson?
 - What does `push()` do?
 - What does `pop()` do?
 - What does `stackTop()` do?
 - What does `pre()` return for `+`, `-`, `*`, `/`, and `#`?
 - What does `isOperand()` return for operands?
 - Why is `#` pushed first?
 - Why is `#` removed at the end?
 - Why do operands go directly to postfix?
 - Why do operators sometimes wait in the stack?
 - Why does `i` not move after some pop operations?
 - What is the time complexity?
 - What is the space complexity?
 - What are the limitations of this beginner version?
-

55. One-Sentence Lesson Summary

Lesson 17 shows how to convert a simple infix expression into postfix form by using a linked-list stack as a temporary waiting area for operators, while operands go directly into the postfix answer.